```
const express = require('express');
const path = require('path');
const { MongoClient } = require('mongodb');
const { Agent, setGlobalDispatcher } = require('undici');
const app = express();

const PORT = process.env.PORT || 10000;
const STATIC_DIR = process.env.STATIC_DIR || 'build';

// Connection pooling/keep-alive to reduce per-segment connection overhead
setGlobalDispatcher(new Agent({ connections: 50, keepAliveTimeout: 30000 }));

app.set('trust proxy', true); // so req.ip reflects the real visitor IP behind Render's proxy

app.use((req, res, next) => {
  res.header('Access-Control-Allow-Origin', '*');
  res.header('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
  res.header('Access-Control-Allow-Headers', 'Content-Type, x-admin-key');
  next();
});
app.use(express.json());

// ---------------- MongoDB ----------------
let usersCollection;
let visitsCollection;
async function initDb() {
  if (!process.env.MONGODB_URI) {
    console.warn('MONGODB_URI not set — user/visit recording disabled');
```

```
    return;
  }
  try {
    const client = new MongoClient(process.env.MONGODB_URI, {
      family: 4, // force IPv4 — fixes SSL alert 80 handshake failures against Atlas on some cloud networks
    });
    await client.connect();
    const db = client.db('worldtv');
    usersCollection = db.collection('users');
    visitsCollection = db.collection('visits');
    console.log('Connected to MongoDB');
  } catch (e) {
    console.error('MongoDB connection failed:', e.message);
  }
}
initDb();

// ---------------- users (email capture) ----------------
app.post('/api/users', async (req, res) => {
  const { email } = req.body;
  if (!email || !/^\S+@\S+\.\S+$/.test(email)) {
    return res.status(400).json({ error: 'Valid email required' });
  }
  if (!usersCollection) {
    return res.status(503).json({ error: 'Database unavailable' });
  }
  try {
```

```
    await usersCollection.updateOne(
      { email },
      { $set: { email, lastSeen: new Date() }, $setOnInsert: { firstSeen: new Date() } },
      { upsert: true }
    );
    res.json({ ok: true });
  } catch (e) {
    console.error('DB error:', e.message);
    res.status(500).json({ error: 'Failed to save' });
  }
});

// ---------------- visit tracking ----------------
async function geolocateIp(ip) {
  if (!ip || ip === '::1' || ip.startsWith('127.') || ip.startsWith('192.168.') || ip.startsWith('10.'))
{
    return { country: 'Local/Unknown', city: 'Local/Unknown' };
  }
  try {
    const cleanIp = ip.replace('::ffff:', '');
    const res = await fetch(`http://ip-
api.com/json/${cleanIp}?fields=status,country,city`);
    const data = await res.json();
    if (data.status === 'success') {
      return { country: data.country || 'Unknown', city: data.city || 'Unknown' };
    }
  } catch (e) {
    console.warn('Geolocation failed:', e.message);
  }
```

```
  return { country: 'Unknown', city: 'Unknown' };
}

app.post('/api/track-visit', async (req, res) => {
  if (!visitsCollection) return res.json({ ok: false });
  try {
    const ip = req.ip || req.headers['x-forwarded-for'] || 'unknown';
    const location = await geolocateIp(ip);
    await visitsCollection.insertOne({
      ip,
      country: location.country,
      city: location.city,
      path: req.body?.path || '/',
      timestamp: new Date(),
    });
    res.json({ ok: true });
  } catch (e) {
    console.error('Visit tracking error:', e.message);
    res.json({ ok: false });
  }
});

// ---------------- admin ----------------
function requireAdmin(req, res, next) {
  const key = req.headers['x-admin-key'];
  if (!process.env.ADMIN_KEY || key !== process.env.ADMIN_KEY) {
    return res.status(401).json({ error: 'Unauthorized' });
  }
```

```
  next();
}

app.get('/api/admin/stats', requireAdmin, async (req, res) => {
  if (!visitsCollection || !usersCollection) {
    return res.status(503).json({ error: 'Database unavailable' });
  }
  try {
    const now = new Date();
    const dayAgo = new Date(now - 24 * 60 * 60 * 1000);
    const weekAgo = new Date(now - 7 * 24 * 60 * 60 * 1000);

    const [visitsAllTime, visitsDay, visitsWeek, recentVisits] = await Promise.all([
      visitsCollection.countDocuments({}),
      visitsCollection.countDocuments({ timestamp: { $gte: dayAgo } }),
      visitsCollection.countDocuments({ timestamp: { $gte: weekAgo } }),
      visitsCollection.find({}).sort({ timestamp: -1 }).limit(100).toArray(),
    ]);

    const [usersAllTime, usersDay, usersWeek, allUsers] = await Promise.all([
      usersCollection.countDocuments({}),
      usersCollection.countDocuments({ firstSeen: { $gte: dayAgo } }),
      usersCollection.countDocuments({ firstSeen: { $gte: weekAgo } }),
      usersCollection.find({}).sort({ firstSeen: -1 }).toArray(),
    ]);

    res.json({
      visits: { allTime: visitsAllTime, day: visitsDay, week: visitsWeek, recent: recentVisits },
```

```
    users: { allTime: usersAllTime, day: usersDay, week: usersWeek, list: allUsers },
   });
  } catch (e) {
   console.error('Admin stats error:', e.message);
   res.status(500).json({ error: 'Failed to load stats' });
  }
});

// ---------------- channel cache ----------------
let channelCache = { data: [], timestamp: 0 };
const CACHE_DURATION = 6 * 60 * 60 * 1000; // 6h

const NAME_MAP = {
  us: 'USA', ca: 'Canada', mx: 'Mexico', gb: 'UK', fr: 'France', de: 'Germany',
  it: 'Italy', es: 'Spain', ru: 'Russia', nl: 'Netherlands', se: 'Sweden', no: 'Norway',
  dk: 'Denmark', fi: 'Finland', pl: 'Poland', tr: 'Turkey', pt: 'Portugal', gr: 'Greece',
  jp: 'Japan', in: 'India', kr: 'South Korea', cn: 'China', tw: 'Taiwan', hk: 'Hong Kong',
  sg: 'Singapore', my: 'Malaysia', ph: 'Philippines', vn: 'Vietnam', th: 'Thailand',
  id: 'Indonesia', pk: 'Pakistan', bd: 'Bangladesh', il: 'Israel', ae: 'UAE',
  sa: 'Saudi Arabia', eg: 'Egypt', za: 'South Africa', ng: 'Nigeria', ke: 'Kenya',
  gh: 'Ghana', br: 'Brazil', ar: 'Argentina', co: 'Colombia', cl: 'Chile', pe: 'Peru',
  au: 'Australia', nz: 'New Zealand',
};
const COUNTRY_CODES = Object.keys(NAME_MAP);

// Verified real YouTube channels (channel ID, not video ID — this always
// resolves to whatever is currently live on that channel, so it never
// goes stale the way a hardcoded video ID would).
```

```
const YOUTUBE_LIVE_CHANNELS = [
 { id: 'yt-citizen-tv-kenya', name: 'Citizen TV Kenya', country: 'Kenya', channelId: 'UChBQgieUidXV1CmDxSdRm3g' },
 { id: 'yt-k24-tv-kenya', name: 'K24 TV', country: 'Kenya', channelId: 'UCt3SE-Mvs3WwP7UW-PiFdqQ' },
 { id: 'yt-aljazeera-english', name: 'Al Jazeera English', country: 'International', channelId: 'UCNye-wNBqNL5ZzHSJj3l8Bg' },
];

function getYoutubeChannels() {
  return YOUTUBE_LIVE_CHANNELS.map((c) => ({
    id: c.id,
    name: c.name,
    logo: '',
    country: c.country,
    category: 'News',
    language: 'Unknown',
    streamUrl: `https://www.youtube.com/embed/live_stream?channel=${c.channelId}`,
  }));
}

function parseM3U(content, countryName) {
  const lines = content.split('\n');
  const channels = [];
  let current = null;

  for (const raw of lines) {
    const line = raw.trim();
    if (line.startsWith('#EXTINF:')) {
```

```
    const nameMatch = line.match(/,(.+)$/);
    const logoMatch = line.match(/tvg-logo="([^"]*)"/);
    const groupMatch = line.match(/group-title="([^"]*)"/);
    const idMatch = line.match(/tvg-id="([^"]*)"/);
    current = {
      id: idMatch && idMatch[1] ? idMatch[1] : `iptv-${countryName}-
${channels.length}`,
      name: nameMatch ? nameMatch[1].trim() : 'Unknown Channel',
      logo: logoMatch && logoMatch[1] ? logoMatch[1] : '',
      country: countryName,
      category: groupMatch && groupMatch[1] ? groupMatch[1] : 'General',
      language: 'Unknown',
      streamUrl: '',
    };
  } else if (line.startsWith('http') && current) {
    current.streamUrl = line;
    channels.push(current);
    current = null;
  }
 }
 return channels;
}

async function fetchCountry(code) {
 const name = NAME_MAP[code] || code.toUpperCase();
 const urls = [
  `https://raw.githubusercontent.com/iptv-org/iptv/master/streams/${code}.m3u`,
  `https://cdn.jsdelivr.net/gh/iptv-org/iptv@master/streams/${code}.m3u`,
```

```
  ];
  for (const url of urls) {
   try {
    const controller = new AbortController();
    const t = setTimeout(() => controller.abort(), 10000);
    const res = await fetch(url, { signal: controller.signal });
    clearTimeout(t);
    if (res.ok) {
     const text = await res.text();
     if (text && text.length > 100) {
      const channels = parseM3U(text, name);
      if (channels.length) return channels;
     }
    }
   } catch (e) {
    console.warn(`[${code}] failed: ${e.message}`);
   }
  }
  return [];
}

async function pool(items, limit, worker) {
  const results = [];
  let i = 0;
  const workers = new Array(Math.min(limit, items.length)).fill(0).map(async () => {
   while (i < items.length) {
    const item = items[i++];
    results.push(...(await worker(item)));
```

```
  }
 });
 await Promise.all(workers);
 return results;
}

async function loadChannels() {
 if (channelCache.data.length && Date.now() - channelCache.timestamp < CACHE_DURATION) {
  return channelCache.data;
 }
 console.log('Refreshing channel cache from iptv-org...');
 const iptvChannels = await pool(COUNTRY_CODES, 8, fetchCountry);
 const channels = [...iptvChannels, ...getYoutubeChannels()];
 if (channels.length) channelCache = { data: channels, timestamp: Date.now() };
 return channelCache.data;
}

app.get('/api/channels', async (req, res) => {
 try {
  const channels = await loadChannels();
  res.json({ channels, count: channels.length, cachedAt: channelCache.timestamp });
 } catch (e) {
  res.status(500).json({ error: 'Failed to load channels', message: e.message });
 }
});

app.get('/api/refresh', async (req, res) => {
```

```
  channelCache = { data: [], timestamp: 0 };
  const channels = await loadChannels();
  res.json({ channels, count: channels.length });
});

// ---------------- stream proxy ----------------
app.get('/api/proxy', async (req, res) => {
  const target = req.query.url;
  if (!target) return res.status(400).send('Missing url param');

  try {
    const upstream = await fetch(target, {
      headers: {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36',
        ...(req.query.referer ? { Referer: req.query.referer } : {}),
      },
    });

    if (!upstream.ok) return res.status(upstream.status).send(`Upstream error: ${upstream.status}`);

    const contentType = upstream.headers.get('content-type') || '';
    const isManifest = target.includes('.m3u8') || contentType.includes('mpegurl');

    if (isManifest) {
      const text = await upstream.text();

      // Some CDNs return an HTML error page with a 200 status instead of
```

```
// a real HTTP error when a stream has expired. Catch that here
// instead of forwarding garbage that causes a "corruption" error
// client-side.
if (!text.trim().startsWith('#EXTM3U')) {
  return res.status(502).send('Upstream returned invalid playlist');
}

const base = new URL(target);
const rewritten = text
  .split('\n')
  .map((line) => {
    const t = line.trim();
    if (!t) return line;
    if (t.startsWith('#')) {
      const uriMatch = t.match(/URI="([^"]+)"/);
      if (uriMatch) {
        const abs = new URL(uriMatch[1], base).href;
        return t.replace(uriMatch[1], `/api/proxy?url=${encodeURIComponent(abs)}`);
      }
      return line;
    }
    const abs = new URL(t, base).href;
    return `/api/proxy?url=${encodeURIComponent(abs)}`;
  })
  .join('\n');

res.set('Content-Type', 'application/vnd.apple.mpegurl');
return res.send(rewritten);
```

```
    }

    res.set('Content-Type', contentType || 'application/octet-stream');
    // Segments are immutable once published — safe to cache briefly to
    // cut repeat-fetch overhead during rebuffers.
    if (target.includes('.ts') || target.includes('.aac')) {
      res.set('Cache-Control', 'public, max-age=30');
    }
    const buffer = Buffer.from(await upstream.arrayBuffer());
    return res.send(buffer);
  } catch (e) {
    console.error('Proxy error:', e.message);
    return res.status(502).send('Proxy fetch failed');
  }
});

// ---------------- serve the built frontend ----------------
const staticPath = path.join(__dirname, '..', STATIC_DIR);
app.use(express.static(staticPath));
app.get('*', (req, res) => res.sendFile(path.join(staticPath, 'index.html')));

app.listen(PORT, () => console.log(` Server running on port ${PORT} `));
import React, { useState, useEffect } from 'react';
import { Channel } from './types/channel.types';
import {
  initializeChannels,
  getCountries,
  getCategories,
```

```
  getChannelsByCountry,
  searchChannels,
  refreshChannels
} from './services/channelService';
import SearchBar from './components/SearchBar';
import Sidebar from './components/Sidebar';
import ChannelGrid from './components/ChannelGrid';
import VideoPlayer from './components/VideoPlayer';
import EmailGate from './components/EmailGate';
import WhatsAppButton from './components/WhatsAppButton';
import { useFavorites } from './hooks/useFavorites';

function App() {
  const [loading, setLoading] = useState(true);
  const [searchQuery, setSearchQuery] = useState('');
  const [selectedCountry, setSelectedCountry] = useState('All');
  const [selectedCategory, setSelectedCategory] = useState('All');
  const [selectedChannel, setSelectedChannel] = useState<Channel | null>(null);
  const [currentChannels, setCurrentChannels] = useState<Channel[]>([]);
  const [hasEmail, setHasEmail] = useState<boolean>(() =>
!!localStorage.getItem('worldtv_email'));

  const countries = getCountries();
  const categories = getCategories();
  const { favorites, toggleFavorite, isFavorite } = useFavorites();

  useEffect(() => {
    fetch('/api/track-visit', {
```

```
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ path: window.location.pathname }),
  }).catch(() => {});
}, []);

useEffect(() => {
  const loadChannels = async () => {
    setLoading(true);
    try {
      console.log('App: Loading channels...');
      await initializeChannels();
      console.log('App: Channels initialized, loading from service...');
      const channels = getChannelsByCountry('All');
      console.log('App: Loaded ' + channels.length + ' channels');
      setCurrentChannels(channels);
    } catch (error) {
      console.error('App: Error loading channels:', error);
    } finally {
      setLoading(false);
    }
  };
  loadChannels();
}, []);

useEffect(() => {
  if (loading) return;
```

```
  let channels: Channel[] = [];
  if (searchQuery) {
    channels = searchChannels(searchQuery);
  } else {
    channels = getChannelsByCountry(selectedCountry);
    if (selectedCategory !== 'All') {
      channels = channels.filter(ch => ch.category === selectedCategory);
    }
  }
  setCurrentChannels(channels);
}, [searchQuery, selectedCountry, selectedCategory, loading]);

const handleCountrySelect = (country: string) => {
  setSelectedCountry(country);
  setSearchQuery('');
};

const handleCategorySelect = (category: string) => {
  setSelectedCategory(category);
  setSearchQuery('');
};

const handleChannelSelect = (channel: Channel) => {
  setSelectedChannel(channel);
};

const handleRandomChannel = () => {
  if (currentChannels.length === 0) return;
```

```
  const randomIndex = Math.floor(Math.random() * currentChannels.length);
  setSelectedChannel(currentChannels[randomIndex]);
};

const handleCloseModal = () => {
  setSelectedChannel(null);
};

const handleRefreshChannels = async () => {
  setLoading(true);
  try {
    console.log('Refreshing channels...');
    await refreshChannels();
    const channels = getChannelsByCountry('All');
    console.log('App: Refreshed ' + channels.length + ' channels');
    setCurrentChannels(channels);
  } catch (error) {
    console.error('App: Error refreshing channels:', error);
  } finally {
    setLoading(false);
  }
};

const handleClearCache = () => {
  if (window.confirm('Clear all cached data and refresh?')) {
    localStorage.removeItem('iptv_channels');
    localStorage.removeItem('favorites');
    alert('Cache cleared! Click Refresh to reload channels.');
```

```
    window.location.reload();
  }
};

if (loading) {
  return (
    <div className="flex items-center justify-center h-screen bg-gray-900">
      <div className="text-center">
        <div className="text-white text-4xl mb-4">Loading</div>
        <div className="text-white text-xl animate-pulse">Loading channels...</div>
        <div className="text-gray-400 text-sm mt-2">This may take a moment</div>
      </div>
    </div>
  );
}

return (
  <div className="flex h-screen bg-gray-900">
    <Sidebar
      selectedCountry={selectedCountry}
      onCountrySelect={handleCountrySelect}
      countries={countries}
      selectedCategory={selectedCategory}
      onCategorySelect={handleCategorySelect}
      categories={categories}
    />
    <div className="flex-1 flex flex-col overflow-hidden">
      <header className="p-4 border-b border-gray-700 bg-gray-800">
```

```
        <div className="flex items-center justify-between mb-3">
          <h1 className="text-white text-2xl font-bold">WorldTV</h1>
          <div className="flex items-center gap-2">
            <span className="text-gray-400 text-sm">
              {currentChannels.length} channels
            </span>
            <button
              onClick={handleClearCache}
              className="bg-red-600 hover:bg-red-700 text-white px-3 py-2 rounded-lg transition flex items-center gap-1 text-sm"
            >
              Clear
            </button>
            <button
              onClick={handleRefreshChannels}
              className="bg-green-600 hover:bg-green-700 text-white px-3 py-2 rounded-lg transition flex items-center gap-1 text-sm"
            >
              Refresh
            </button>
            <button
              onClick={handleRandomChannel}
              className="bg-blue-600 hover:bg-blue-700 text-white px-3 py-2 rounded-lg transition flex items-center gap-1 text-sm"
            >
              Random
            </button>
          </div>
        </div>
```

```
        <SearchBar value={searchQuery} onChange={setSearchQuery} />
      </header>
      <main className="flex-1 overflow-y-auto p-4">
        <ChannelGrid
          channels={currentChannels}
          onChannelSelect={handleChannelSelect}
          favorites={favorites}
          onToggleFavorite={toggleFavorite}
          isFavorite={isFavorite}
        />
      </main>
    </div>
    {selectedChannel && (
      <div
        className="fixed inset-0 bg-black bg-opacity-75 flex items-center justify-center z-50 p-4"
        onClick={handleCloseModal}
      >
        <div
          className="bg-gray-800 rounded-lg max-w-4xl w-full max-h-[90vh] overflow-y-auto"
          onClick={(e) => e.stopPropagation()}
        >
          <div className="flex justify-between items-start p-4 border-b border-gray-700">
            <div className="flex-1">
              <h2 className="text-white text-xl font-bold">{selectedChannel.name}</h2>
              <div className="flex items-center gap-3 mt-1 flex-wrap">
                <span className="text-gray-400 text-sm">{selectedChannel.country}</span>
```

```
            <span className="text-blue-400 text-sm bg-blue-900/30 px-2 py-0.5 rounded">
              {selectedChannel.category}
            </span>
            <span className="text-gray-500 text-sm">{selectedChannel.language}</span>
            <button
              onClick={() => toggleFavorite(selectedChannel.id)}
              className={isFavorite(selectedChannel.id) ? 'text-xl transition text-yellow-400' : 'text-xl transition text-gray-500 hover:text-yellow-400'}
            >
              {isFavorite(selectedChannel.id) ? 'Fav' : 'Not fav'}
            </button>
          </div>
        </div>
        <button
          onClick={handleCloseModal}
          className="text-gray-400 hover:text-white text-2xl leading-none ml-4"
        >
          Close
        </button>
      </div>
      <div className="p-4 relative">
        {!hasEmail && <EmailGate onSubmit={() => setHasEmail(true)} />}
        {hasEmail && (
          <VideoPlayer
            streamUrl={selectedChannel.streamUrl}
            channelName={selectedChannel.name}
          />
```

```
          )}
        </div>
        <div className="p-4 border-t border-gray-700 bg-gray-800/50">
         <div className="flex items-center justify-between text-sm">
          <div className="text-gray-400 truncate">
           <span className="font-medium">Stream URL:</span>{' '}
           <span className="text-gray-500 truncate inline-block max-w-xs">
            {selectedChannel.streamUrl}
           </span>
          </div>
          <button
           onClick={() => window.open(selectedChannel.streamUrl, '_blank')}
           className="bg-blue-600 hover:bg-blue-700 text-white px-4 py-2 rounded-lg
transition flex items-center gap-2"
          >
           Open in new tab
          </button>
         </div>
        </div>
       </div>
      </div>
    )}
    <WhatsAppButton />
   </div>
  );
}

export default App;
```

```
import React from 'react';
import { Channel } from '../types/channel.types';

interface ChannelCardProps {
  channel: Channel;
  onClick: () => void;
  isFavorite: boolean;
  onToggleFavorite: () => void;
}

const ChannelCard: React.FC<ChannelCardProps> = ({
  channel,
  onClick,
  isFavorite,
  onToggleFavorite
}) => {
  return (
    <div className="relative">
      <div
        className="bg-gray-800 rounded-lg p-3 hover:bg-gray-700 transition-all cursor-pointer border border-gray-700 hover:border-blue-500 hover:scale-105 transform duration-200"
        onClick={onClick}
      >
        <div className="flex flex-col items-center">
          <div className="w-20 h-20 bg-gray-700 rounded-lg flex items-center justify-center mb-2 overflow-hidden">
            <img
```

```
          src={channel.logo}
          alt={channel.name}
          className="w-full h-full object-contain p-2"
          onError={(e) => {
            (e.target as HTMLImageElement).src = `data:image/svg+xml,<svg xmlns="http://www.w3.org/2000/svg" width="80" height="80" viewBox="0 0 80 80"><rect width="80" height="80" fill="%23333"/><text x="40" y="45" text-anchor="middle" fill="%23666" font-size="12">${channel.name.substring(0, 3)}</text></svg>`;
          }}
        />
      </div>
      <h3 className="text-white text-sm text-center font-medium truncate w-full max-w-[120px]">
        {channel.name}
      </h3>
      <div className="flex items-center gap-2 mt-1">
        <span className="text-gray-400 text-xs">{channel.country}</span>
        <span className="text-xs text-blue-400 bg-blue-900/30 px-2 py-0.5 rounded">
          {channel.category}
        </span>
      </div>
      <span className="text-xs text-gray-500 mt-1">{channel.language}</span>
    </div>
  </div>

  {/* Favorite Button */}
  <button
    onClick={(e) => {
```

```
        e.stopPropagation();
        onToggleFavorite();
      }}
      className="absolute top-2 right-2 text-xl transition hover:scale-110"
    >
      {isFavorite ? '⭐' : '☆'}
    </button>
  </div>
 );
};

export default ChannelCard;
```